

## Phần 6: Các Tính năng Nâng cao

### 6.1 Locking trong JPA

JPA hỗ trợ hai loại locking để xử lý concurrent access:

#### 6.1.1 Optimistic Locking

Optimistic Locking là một chiến lược kiểm soát đồng thời (concurrency control) giả định rằng các xung đột (conflict) giữa các transaction là hiếm khi xảy ra. Thay vì khóa bản ghi ngay lập tức, nó sử dụng một cơ chế "versioning" (phiên bản) để phát hiện xem một bản ghi có bị thay đổi bởi một transaction khác trong quá trình đọc và cập nhật hay không. Nếu có xung đột, transaction hiện tại sẽ bị rollback và ném ra ngoại lệ.

Java

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Sử dụng
    GenerationType.IDENTITY cho ID tự tăng.
    private Long id;

    private String name;

    @Version // Đánh dấu thuộc tính này là trường phiên bản cho Optimistic
    Locking.
    private Long version; // Hibernate/JPA tự động quản lý giá trị của trường
    này.

    // constructors, getters, setters
}

// Repository
public interface UserRepository extends JpaRepository<User, Long> {

    @Lock(LockModeType.OPTIMISTIC) // Yêu cầu Optimistic Lock khi đọc Entity.
    Optional<User> findById(Long id);

    @Lock(LockModeType.OPTIMISTIC_FORCE_INCREMENT) // Yêu cầu Optimistic Lock
    và buộc tăng version ngay cả khi không có thay đổi dữ liệu.
    @Query("SELECT u FROM User u WHERE u.email = :email")
    User findByEmailWithOptimisticLock(@Param("email") String email);
}
```

```
// Service
@Service
@Transactional // Đảm bảo phương thức này chạy trong một transaction.
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public void updateUser(Long userId, String newName) {
        try {
            // Tìm User. Nếu không tìm thấy, ném ngoại lệ.
            User user = userRepository.findById(userId).orElseThrow(() -> new
EntityNotFoundException("User not found with ID: " + userId));
            user.setName(newName); // Thay đổi dữ liệu.
            userRepository.save(user); // Lưu User. Nếu version không khớp,
OptimisticLockingFailureException sẽ được ném.
        } catch (OptimisticLockingFailureException e) {
            // Xử lý trường hợp có sự thay đổi đồng thời.
            throw new ConcurrentModificationException("User was modified by
another transaction. Please try again.", e);
        }
    }
}
```

## Giải thích:

- **@Version** : Annotation này được đặt trên một thuộc tính (thường là `Long` hoặc `Integer` ) trong Entity. JPA/Hibernate sẽ tự động tăng giá trị của trường này mỗi khi Entity được cập nhật. Khi một Entity được tải, giá trị `version` của nó cũng được tải. Khi Entity được lưu lại, JPA sẽ kiểm tra xem giá trị `version` trong cơ sở dữ liệu có khớp với giá trị `version` đã tải hay không. Nếu không khớp, điều đó có nghĩa là một transaction khác đã thay đổi bản ghi, và `OptimisticLockingFailureException` sẽ được ném ra.
- **@Lock(LockModeType.OPTIMISTIC)** : Khi bạn sử dụng annotation này trên một phương thức truy vấn, JPA sẽ đảm bảo rằng khi Entity được tải, giá trị `version` của nó sẽ được kiểm tra khi Entity đó được cập nhật hoặc xóa trong cùng một transaction. Nó không khóa bản ghi trong cơ sở dữ liệu.
- **@Lock(LockModeType.OPTIMISTIC\_FORCE\_INCREMENT)** : Tương tự như `OPTIMISTIC` , nhưng nó buộc giá trị `version` phải tăng lên ngay cả khi không có thay đổi nào được thực hiện trên các thuộc tính khác của Entity. Điều này hữu ích khi bạn muốn đảm bảo rằng một

bản ghi được đánh dấu là đã được truy cập hoặc xử lý, ngay cả khi không có thay đổi dữ liệu thực tế.

- **OptimisticLockingFailureException** : Ngoại lệ này được ném ra khi JPA phát hiện một xung đột Optimistic Lock (tức là bản ghi đã bị thay đổi bởi một transaction khác).
- **Lợi ích**: Hiệu suất cao hơn so với Pessimistic Locking vì không cần khóa cơ sở dữ liệu, cho phép nhiều transaction đọc cùng một bản ghi đồng thời. Thích hợp cho các ứng dụng có tỷ lệ xung đột thấp.

### 6.1.2 Pessimistic Locking

Pessimistic Locking là một chiến lược kiểm soát đồng thời giả định rằng các xung đột là phổ biến. Nó khóa bản ghi trong cơ sở dữ liệu ngay lập tức khi một transaction bắt đầu truy cập nó, ngăn chặn các transaction khác đọc hoặc sửa đổi bản ghi đó cho đến khi khóa được giải phóng. Điều này đảm bảo tính toàn vẹn dữ liệu cao nhưng có thể ảnh hưởng đến hiệu suất do giảm tính đồng thời.

Java

```
public interface UserRepository extends JpaRepository<User, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE) // Yêu cầu khóa ghi (write lock)
    trên bản ghi. Các transaction khác không thể đọc hoặc ghi.
    @Query("SELECT u FROM User u WHERE u.id = :id")
    User findByIdWithPessimisticLock(@Param("id") Long id);

    @Lock(LockModeType.PESSIMISTIC_READ) // Yêu cầu khóa đọc (read lock) trên
    bản ghi. Các transaction khác có thể đọc nhưng không thể ghi.
    List<User> findByStatus(String status);

    @Lock(LockModeType.PESSIMISTIC_FORCE_INCREMENT) // Tương tự
    PESSIMISTIC_WRITE nhưng cũng buộc tăng version (nếu có).
    @Query("SELECT u FROM User u WHERE u.email = :email")
    User findByEmailWithPessimisticForceLock(@Param("email") String email);
}
```

#### Giải thích:

- **@Lock** : Annotation này được sử dụng trên các phương thức repository để chỉ định loại khóa Pessimistic Lock.

- **LockModeType.PESSIMISTIC\_WRITE** : Đây là loại khóa mạnh nhất. Khi một transaction có được khóa `PESSIMISTIC_WRITE` trên một bản ghi, không có transaction nào khác có thể đọc hoặc ghi bản ghi đó cho đến khi khóa được giải phóng. Điều này ngăn chặn cả "dirty reads", "non-repeatable reads" và "phantom reads".
- **LockModeType.PESSIMISTIC\_READ** : Loại khóa này cho phép nhiều transaction đọc cùng một bản ghi đồng thời, nhưng ngăn chặn bất kỳ transaction nào khác ghi vào bản ghi đó. Nó ngăn chặn "dirty reads" và "non-repeatable reads".
- **LockModeType.PESSIMISTIC\_FORCE\_INCREMENT** : Tương tự như `PESSIMISTIC_WRITE` , nhưng nó cũng buộc giá trị `version` (nếu có trường `@Version` trong Entity) phải tăng lên, ngay cả khi không có thay đổi dữ liệu thực tế. Điều này hữu ích khi bạn muốn đảm bảo rằng một bản ghi được đánh dấu là đã được xử lý bởi một transaction, ngay cả khi không có thay đổi dữ liệu.
- **Lợi ích:** Đảm bảo tính toàn vẹn dữ liệu cao nhất trong môi trường đồng thời. Thích hợp cho các trường hợp mà xung đột dữ liệu là phổ biến và cần được ngăn chặn một cách nghiêm ngặt.
- **Nhược điểm:** Có thể làm giảm hiệu suất của ứng dụng do giảm tính đồng thời, vì các transaction phải chờ đợi để có được khóa. Nên sử dụng cẩn thận và chỉ khi thực sự cần thiết.

## 6.2 JPA Callbacks và Entity Listeners

### 6.2.1 JPA Lifecycle Callbacks

JPA cung cấp các callback methods cho phép bạn thực hiện các hành động tùy chỉnh tại các thời điểm khác nhau trong vòng đời của một Entity (ví dụ: trước khi lưu, sau khi tải, trước khi cập nhật, v.v.). Các callback này được đánh dấu bằng các annotation cụ thể.

Java

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

private String name;
private Date createdAt;
private Date updatedAt;

@PrePersist // Được gọi trước khi Entity được lưu lần đầu tiên (INSERT).
protected void onCreate() {
    createdAt = new Date();
    updatedAt = new Date();
    System.out.println("PrePersist: User will be created.");
}

@PreUpdate // Được gọi trước khi Entity được cập nhật (UPDATE).
protected void onUpdate() {
    updatedAt = new Date();
    System.out.println("PreUpdate: User will be updated.");
}

@PostLoad // Được gọi sau khi Entity được tải từ cơ sở dữ liệu.
protected void onLoad() {
    System.out.println("PostLoad: User loaded: " + name);
}

@PreRemove // Được gọi trước khi Entity bị xóa.
protected void onRemove() {
    System.out.println("PreRemove: User will be removed: " + name);
}

@PostPersist // Được gọi sau khi Entity đã được lưu lần đầu tiên.
protected void onPostPersist() {
    System.out.println("PostPersist: User persisted with ID: " + id);
}

@PostUpdate // Được gọi sau khi Entity đã được cập nhật.
protected void onPostUpdate() {
    System.out.println("PostUpdate: User updated: " + name);
}

@PostRemove // Được gọi sau khi Entity đã bị xóa.
protected void onPostRemove() {
    System.out.println("PostRemove: User removed: " + name);
}

// getters and setters
}

```

**Giải thích:**

- **@PrePersist** : Phương thức được đánh dấu bằng `@PrePersist` sẽ được gọi ngay trước khi một Entity mới được lưu vào cơ sở dữ liệu (thao tác `persist` ). Thường được sử dụng để thiết lập các giá trị mặc định như `createdDate` .
- **@PreUpdate** : Phương thức này được gọi ngay trước khi một Entity hiện có được cập nhật trong cơ sở dữ liệu (thao tác `merge` ). Thường được sử dụng để cập nhật `updatedAt` .
- **@PostLoad** : Được gọi sau khi một Entity đã được tải từ cơ sở dữ liệu hoặc sau khi một truy vấn đã được thực thi và kết quả được ánh xạ vào Entity. Thường dùng để thực hiện các thao tác sau khi dữ liệu đã sẵn sàng.
- **@PreRemove** : Được gọi ngay trước khi một Entity bị xóa khỏi cơ sở dữ liệu.
- **@PostPersist** : Được gọi sau khi một Entity mới đã được lưu vào cơ sở dữ liệu. Tại thời điểm này, Entity đã có ID được sinh ra.
- **@PostUpdate** : Được gọi sau khi một Entity hiện có đã được cập nhật trong cơ sở dữ liệu.
- **@PostRemove** : Được gọi sau khi một Entity đã bị xóa khỏi cơ sở dữ liệu.
- **Vị trí**: Các phương thức callback có thể là `public` hoặc `protected` , không cần tham số và không được trả về giá trị. Chúng có thể được đặt trong lớp Entity hoặc trong một lớp Listener riêng biệt.

### 6.2.2 Entity Listeners

Entity Listeners là các lớp riêng biệt chứa các phương thức callback của JPA. Điều này cho phép bạn tách biệt logic callback khỏi lớp Entity, giúp Entity gọn gàng hơn và cho phép tái sử dụng logic callback trên nhiều Entity khác nhau. Entity Listener được đăng ký với Entity bằng annotation `@EntityListeners` .

Java

```
// Listener class: Lớp chứa các phương thức callback.
public class AuditingEntityListener {

    @PrePersist // Callback trước khi Entity được lưu lần đầu.
    public void prePersist(Object entity) {
        if (entity instanceof Auditable) { // Kiểm tra xem Entity có
```

*implement interface Auditable không.*

```
Auditable auditable = (Auditable) entity;  
auditable.setCreatedDate(new Date()); // Thiết lập ngày tạo.  
auditable.setCreatedBy(getCurrentUser()); // Thiết lập người tạo.
```

```
}
```

```
}
```

*@PreUpdate // Callback trước khi Entity được cập nhật.*

```
public void preUpdate(Object entity) {  
    if (entity instanceof Auditable) {  
        Auditable auditable = (Auditable) entity;  
        auditable.setUpdatedDate(new Date()); // Thiết lập ngày cập nhật.  
        auditable.setUpdatedBy(getCurrentUser()); // Thiết lập người cập
```

*nhật.*

```
}
```

```
}
```

```
private String getCurrentUser() {
```

*// Logic để lấy người dùng hiện tại từ Security Context (ví dụ: Spring Security).*

```
    return "current_user"; // Trả về một giá trị mẫu.
```

```
}
```

```
}
```

*// Auditable interface: Một interface để các Entity có thể implement, đảm bảo chúng có các thuộc tính auditing.*

```
public interface Auditable {  
    void setCreatedDate(Date createdDate);  
    void setCreatedBy(String createdBy);  
    void setUpdatedDate(Date updatedDate);  
    void setUpdatedBy(String updatedBy);  
}
```

*// Entity with listener: Lớp Entity sử dụng Entity Listener.*

```
@Entity
```

```
@EntityListeners(AuditingEntityListener.class) // Đăng ký
```

*AuditingEntityListener cho Entity này.*

```
public class User implements Auditable {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    private Date createdDate;
```

```
    private String createdBy;
```

```
    private Date updatedDate;
```

```
    private String updatedBy;
```

```

// implement Auditable methods: Triển khai các phương thức của interface Auditable.
@Override
public void setCreatedDate(Date createdDate) {
    this.createdDate = createdDate;
}

@Override
public void setCreatedBy(String createdBy) {
    this.createdBy = createdBy;
}

@Override
public void setUpdatedDate(Date updatedDate) {
    this.updatedDate = updatedDate;
}

@Override
public void setUpdatedBy(String updatedBy) {
    this.updatedBy = updatedBy;
}
// other implementations...
}

```

### Giải thích:

- **Tách biệt mối quan tâm:** Entity Listeners giúp tách biệt logic xử lý vòng đời Entity khỏi chính lớp Entity, làm cho code sạch hơn và dễ bảo trì hơn.
- **Tái sử dụng:** Một Entity Listener có thể được sử dụng bởi nhiều Entity khác nhau, đặc biệt hữu ích cho các chức năng chung như auditing (ghi lại thông tin người tạo, ngày tạo, người cập nhật, ngày cập nhật).
- **@EntityListeners(AuditingEntityListener.class)** : Annotation này được đặt trên lớp Entity để chỉ định một hoặc nhiều lớp Listener sẽ lắng nghe các sự kiện vòng đời của Entity đó.
- **Phương thức Callback:** Các phương thức trong lớp Listener cũng được đánh dấu bằng các annotation callback tương tự như khi đặt trực tiếp trong Entity (ví dụ: `@PrePersist` , `@PreUpdate` ). Các phương thức này nhận một tham số kiểu `Object` (đại diện cho Entity đang được xử lý) hoặc kiểu cụ thể của Entity đó.

## 6.3 Spring Data JPA Auditing



Spring Data JPA cung cấp một cơ chế auditing tự động, giúp bạn dễ dàng theo dõi thông tin về người tạo, ngày tạo, người cập nhật cuối cùng và ngày cập nhật cuối cùng cho các Entity. Điều này rất hữu ích cho việc kiểm toán và theo dõi lịch sử thay đổi dữ liệu.

### 6.3.1 Enable Auditing

Để kích hoạt tính năng auditing trong Spring Data JPA, bạn cần thêm annotation

`@EnableJpaAuditing` vào một lớp cấu hình Spring (thường là lớp `Application` chính hoặc một lớp cấu hình riêng).

Java

```
@Configuration // Đánh dấu đây là một lớp cấu hình Spring.
@EnableJpaAuditing // Kích hoạt tính năng auditing của Spring Data JPA.
public class JpaConfig {

    @Bean // Định nghĩa một Spring Bean.
    public AuditorAware<String> auditorProvider() {
        // AuditorAware là một interface mà Spring Data JPA sử dụng để lấy
        // thông tin về người dùng hiện tại.
        return () -> {
            // Logic để lấy người dùng hiện tại từ Security Context (ví dụ:
            // Spring Security).
            // Trả về Optional.of("current_user") làm ví dụ. Trong thực tế,
            // bạn sẽ lấy từ Authentication object.
            return Optional.of("current_user");
        };
    }
}
```

#### Giải thích:

- `@EnableJpaAuditing` : Annotation này được đặt trên một lớp cấu hình Spring để kích hoạt tính năng auditing. Khi được kích hoạt, Spring Data JPA sẽ tự động điền các trường auditing ( `@CreatedBy` , `@CreatedDate` , `@LastModifiedBy` , `@LastModifiedDate` ) khi Entity được lưu hoặc cập nhật.
- `AuditorAware<T>` : Đây là một interface mà bạn cần triển khai để cung cấp thông tin về người dùng hiện tại (auditor). Spring Data JPA sẽ gọi phương thức `getCurrentAuditor()` của interface này để lấy thông tin người dùng khi cần điền vào các trường `@CreatedBy`

hoặc `@LastModifiedBy` . Kiểu `T` là kiểu dữ liệu của ID người dùng (ví dụ: `String` cho username, `Long` cho user ID).

- `@Bean` : Định nghĩa `AuditorAware` như một Spring Bean để Spring có thể tự động inject nó vào các thành phần cần thiết.

### 6.3.2 Auditable Entity

Để các Entity của bạn có thể sử dụng tính năng auditing, bạn cần đánh dấu các thuộc tính tương ứng bằng các annotation auditing của Spring Data JPA.

Java

```
@Entity
@EntityListeners(AuditingEntityListener.class) // Vẫn có thể sử dụng
EntityListeners nếu muốn tùy chỉnh thêm.
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @CreatedDate // Tự động điền ngày/giờ tạo Entity.
    private LocalDateTime createdAt;

    @LastModifiedDate // Tự động điền ngày/giờ cập nhật Entity lần cuối.
    private LocalDateTime lastModifiedDate;

    @CreatedBy // Tự động điền thông tin người tạo Entity (lấy từ
AuditorAware).
    private String createdBy;

    @LastModifiedBy // Tự động điền thông tin người cập nhật Entity lần cuối
(lấy từ AuditorAware).
    private String lastModifiedBy;

    // getters and setters
}

// Hoặc sử dụng base class để tái sử dụng các thuộc tính auditing cho nhiều
Entity.
@MappedSuperclass // Đánh dấu lớp này là một superclass mà các Entity khác có
thể kế thừa.
@EntityListeners(AuditingEntityListener.class) // Có thể đặt listener ở đây
```

để áp dụng cho tất cả các lớp con.

```
public abstract class BaseEntity {

    @CreatedDate
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime lastModifiedDate;

    @CreatedBy
    private String createdBy;

    @LastModifiedBy
    private String lastModifiedBy;

    // getters and setters
}

@Entity
public class User extends BaseEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    // other fields
}
```

## Giải thích:

- **@CreatedDate** : Tự động điền ngày và giờ hiện tại khi Entity được tạo lần đầu tiên. Kiểu dữ liệu thường là `java.util.Date` , `java.time.LocalDate` , `java.time.LocalDateTime` , v.v.
- **@LastModifiedDate** : Tự động điền ngày và giờ hiện tại mỗi khi Entity được cập nhật. Kiểu dữ liệu tương tự như `@CreatedDate` .
- **@CreatedBy** : Tự động điền thông tin người dùng (được cung cấp bởi `AuditorAware` ) khi Entity được tạo. Kiểu dữ liệu phải khớp với kiểu trả về của `AuditorAware` .
- **@LastModifiedBy** : Tự động điền thông tin người dùng (được cung cấp bởi `AuditorAware` ) khi Entity được cập nhật. Kiểu dữ liệu tương tự như `@CreatedBy` .
- **@MappedSuperclass** : Annotation này được sử dụng trên một lớp cơ sở (base class) để các thuộc tính của nó có thể được kế thừa bởi các Entity con mà không cần lớp cơ sở đó.

phải là một Entity riêng biệt. Điều này rất hữu ích để định nghĩa các thuộc tính chung như auditing cho nhiều Entity.

- **Kết hợp với `EntityListeners`** : Mặc dù Spring Data JPA Auditing tự động điền các trường, bạn vẫn có thể sử dụng `@EntityListeners` để thêm các logic tùy chỉnh khác trong quá trình auditing hoặc các sự kiện vòng đời khác.

## 6.4 Custom Repository Implementations

Spring Data JPA cung cấp các Repository interface mạnh mẽ, nhưng đôi khi bạn cần thực hiện các thao tác phức tạp hơn không thể được xử lý chỉ bằng cách định nghĩa phương thức trong interface (ví dụ: các truy vấn động phức tạp, các thao tác batch, hoặc tích hợp với các thư viện khác). Trong những trường hợp này, bạn có thể tạo các implementation tùy chỉnh cho Repository của mình.

### 6.4.1 Custom Repository Interface

Đầu tiên, bạn định nghĩa một interface mới chứa các phương thức tùy chỉnh mà bạn muốn thêm vào Repository của mình. Interface này không cần kế thừa bất kỳ interface Spring Data JPA nào.

Java

```
public interface CustomUserRepository {
    List<User> findUsersWithCustomLogic(String criteria);
    void bulkUpdateUserStatus(List<Long> userIds, String status);
    Page<User> findUsersWithComplexSearch(UserSearchCriteria criteria,
    Pageable pageable);
}
```

#### Giải thích:

- **Mục đích:** Interface này chứa các phương thức mà bạn không thể (hoặc không muốn) định nghĩa bằng cách sử dụng Query Methods của Spring Data JPA hoặc `@Query` đơn giản. Các phương thức này sẽ được triển khai thủ công.
- **Không kế thừa:** Interface này không cần kế thừa `JpaRepository` hay bất kỳ interface Spring Data JPA nào khác. Nó chỉ định nghĩa các phương thức tùy chỉnh của bạn.

- **Ví dụ:** Các phương thức như `findUsersWithCustomLogic` (có thể sử dụng Criteria API hoặc Querydsl), `bulkUpdateUserStatus` (thực hiện các thao tác batch), hoặc `findUsersWithComplexSearch` (kết hợp nhiều tiêu chí tìm kiếm và phân trang) là những ví dụ điển hình cho các phương thức tùy chỉnh.

### 6.4.2 Custom Repository Implementation

Tiếp theo, bạn cần tạo một lớp triển khai (implementation class) cho interface tùy chỉnh này. Lớp này sẽ chứa logic nghiệp vụ cho các phương thức tùy chỉnh. Tên của lớp implementation phải tuân theo quy ước của Spring Data JPA: tên của interface repository chính (ví dụ: `UserRepository`) cộng với hậu tố `Impl` (ví dụ: `UserRepositoryImpl`).

Java

```
@Repository // Đánh dấu lớp này là một Spring component để Spring có thể quét và quản lý.
public class CustomUserRepositoryImpl implements CustomUserRepository {

    @PersistenceContext // Inject EntityManager để tương tác trực tiếp với JPA.
    private EntityManager entityManager;

    @Override
    public List<User> findUsersWithCustomLogic(String criteria) {
        // Sử dụng JPA Criteria API để xây dựng truy vấn động.
        CriteriaBuilder cb = entityManager.getCriteriaBuilder();
        CriteriaQuery<User> query = cb.createQuery(User.class);
        Root<User> root = query.from(User.class);

        // Logic điều kiện phức tạp.
        Predicate predicate = cb.conjunction(); // Bắt đầu với một predicate luôn đúng.

        if (criteria != null && !criteria.isEmpty()) {
            predicate = cb.and(predicate,
                cb.or(
                    cb.like(cb.lower(root.get("name")), "%" +
                        criteria.toLowerCase() + "%"), // Tìm kiếm theo tên (không phân biệt hoa thường).
                    cb.like(cb.lower(root.get("email")), "%" +
                        criteria.toLowerCase() + "%") // Hoặc tìm kiếm theo email.
                )
            );
        }
    }
}
```

```

        query.where(predicate); // Áp dụng điều kiện vào truy vấn.
        query.orderBy(cb.asc(root.get("name"))); // Sắp xếp kết quả theo tên.

        return entityManager.createQuery(query).getResultList(); // Thực thi
truy vấn và trả về danh sách User.
    }

    @Override
    @Transactional // Đảm bảo phương thức này chạy trong một transaction.
    public void bulkUpdateUserStatus(List<Long> userIds, String status) {
        if (userIds == null || userIds.isEmpty()) {
            return;
        }

        // Sử dụng JPQL để thực hiện cập nhật hàng loạt.
        String jpql = "UPDATE User u SET u.status = :status WHERE u.id IN
:ids";
        entityManager.createQuery(jpql)
            .setParameter("status", status)
            .setParameter("ids", userIds)
            .executeUpdate(); // Thực thi truy vấn cập nhật.
    }

    @Override
    public Page<User> findUsersWithComplexSearch(UserSearchCriteria criteria,
Pageable pageable) {
        CriteriaBuilder cb = entityManager.getCriteriaBuilder();

        // 1. Truy vấn đếm tổng số bản ghi (cho phân trang).
        CriteriaQuery<Long> countQuery = cb.createQuery(Long.class);
        Root<User> countRoot = countQuery.from(User.class);
        countQuery.select(cb.count(countRoot));
        countQuery.where(buildPredicates(cb, countRoot, criteria)); // Áp
dụng các điều kiện tìm kiếm.
        Long total = entityManager.createQuery(countQuery).getSingleResult();

        // 2. Truy vấn lấy dữ liệu thực tế.
        CriteriaQuery<User> dataQuery = cb.createQuery(User.class);
        Root<User> dataRoot = dataQuery.from(User.class);
        dataQuery.where(buildPredicates(cb, dataRoot, criteria)); // Áp dụng
các điều kiện tìm kiếm.

        // Áp dụng sắp xếp (sorting).
        if (pageable.getSort().isSorted()) {
            List<Order> orders = new ArrayList<>();
            for (Sort.Order order : pageable.getSort()) {
                if (order.isAscending()) {

```

```

        orders.add(cb.asc(dataRoot.get(order.getProperty())));
    } else {
        orders.add(cb.desc(dataRoot.get(order.getProperty())));
    }
}
dataQuery.orderBy(orders);
}

// Thực thi truy vấn với phân trang.
List<User> content = entityManager.createQuery(dataQuery)
    .setFirstResult((int) pageable.getOffset()) // Vị trí bắt đầu của
kết quả.
    .setMaxResults(pageable.getPageSize()) // Số lượng kết quả tối đa
trên một trang.
    .getResultList();

// Trả về PageImpl chứa dữ liệu, thông tin phân trang và tổng số bản
ghi.
return new PageImpl<>(content, pageable, total);
}

// Phương thức helper để xây dựng các Predicate cho Criteria API.
private Predicate buildPredicates(CriteriaBuilder cb, Root<User> root,
UserSearchCriteria criteria) {
    Predicate predicate = cb.conjunction();

    if (criteria.getName() != null) {
        predicate = cb.and(predicate,
            cb.like(cb.lower(root.get("name")), "%" +
criteria.getName().toLowerCase() + "%"));
    }

    if (criteria.getEmail() != null) {
        predicate = cb.and(predicate,
            cb.equal(root.get("email"), criteria.getEmail()));
    }

    if (criteria.getMinAge() != null) {
        predicate = cb.and(predicate,
            cb.greaterThanOrEqualTo(root.get("age"),
criteria.getMinAge()));
    }

    if (criteria.getMaxAge() != null) {
        predicate = cb.and(predicate,
            cb.lessThanOrEqualTo(root.get("age"), criteria.getMaxAge()));
    }
}

```

```

        return predicate;
    }
}

```

## Giải thích:

- **@Repository** : Đánh dấu lớp này là một Spring component, cho phép Spring tự động quét và quản lý nó. Điều này cũng giúp Spring Data JPA tìm thấy implementation tùy chỉnh này.
- **@PersistenceContext** : Inject `EntityManager` vào lớp implementation. `EntityManager` là giao diện chính để tương tác với JPA, cho phép bạn thực hiện các thao tác như tạo truy vấn, lưu, cập nhật, xóa Entity.
- **EntityManager.getCriteriaBuilder()** : Lấy về một instance của `CriteriaBuilder` , được sử dụng để xây dựng các truy vấn Criteria API.
- **CriteriaBuilder.createQuery(User.class)** : Tạo một `CriteriaQuery` cho Entity `User` .
- **query.from(User.class)** : Định nghĩa `Root` của truy vấn, chỉ ra Entity mà truy vấn sẽ bắt đầu từ đó.
- **buildPredicates(...)** : Một phương thức helper để xây dựng các điều kiện `WHERE` ( `Predicate` ) dựa trên các tiêu chí tìm kiếm. Điều này giúp tái sử dụng logic xây dựng điều kiện cho cả truy vấn đếm và truy vấn dữ liệu.
- **entityManager.createQuery(query).getResultList()** : Thực thi truy vấn Criteria API và trả về danh sách kết quả.
- **entityManager.createQuery(jpql).setParameter(...).executeUpdate()** : Thực thi một truy vấn JPQL để cập nhật hoặc xóa dữ liệu. `executeUpdate()` trả về số lượng bản ghi bị ảnh hưởng.
- **PageImpl<>(content, pageable, total)** : Khi thực hiện phân trang thủ công, bạn cần tạo một đối tượng `PageImpl` để trả về kết quả. Nó chứa danh sách dữ liệu ( `content` ), thông tin phân trang ( `pageable` ), và tổng số bản ghi ( `total` ).
- **@Transactional** : Các phương thức thực hiện thay đổi dữ liệu (như `bulkUpdateUserStatus` ) cần được đánh dấu `@Transactional` để đảm bảo tính toàn vẹn dữ



liệu. Spring sẽ quản lý transaction cho các phương thức này.

### 6.4.3 Combine với Standard Repository

Để kết hợp các phương thức tùy chỉnh của bạn với các phương thức tiêu chuẩn của Spring Data JPA (như `save`, `findById`, `findAll`), bạn chỉ cần làm cho interface repository chính của bạn kế thừa cả `JpaRepository` và interface tùy chỉnh của bạn.

Java

```
public interface UserRepository extends JpaRepository<User, Long>,
CustomUserRepository {
    // Standard Spring Data JPA methods + custom methods
    List<User> findByStatus(String status);

    // Các phương thức tùy chỉnh từ CustomUserRepository (ví dụ:
    findUsersWithCustomLogic) sẽ tự động có sẵn.
}
```

#### Giải thích:

- **Kế thừa đa interface:** Bằng cách kế thừa cả `JpaRepository` và `CustomUserRepository`, interface `UserRepository` của bạn sẽ có tất cả các phương thức CRUD tiêu chuẩn từ `JpaRepository` và tất cả các phương thức tùy chỉnh mà bạn đã định nghĩa trong `CustomUserRepository`.
- **Tự động phát hiện:** Spring Data JPA sẽ tự động phát hiện lớp `CustomUserRepositoryImpl` (do quy ước đặt tên `[TênRepository]Impl`) và liên kết nó với `UserRepository`. Khi bạn inject `UserRepository` vào các lớp Service, bạn có thể gọi cả các phương thức tiêu chuẩn và các phương thức tùy chỉnh.
- **Lợi ích:** Cách tiếp cận này cho phép bạn mở rộng chức năng của Spring Data JPA mà không làm mất đi sự tiện lợi của các phương thức được tạo tự động. Nó giữ cho code của bạn sạch sẽ và dễ quản lý, tách biệt các truy vấn phức tạp ra khỏi các phương thức repository đơn giản.

### 6.5 Querydsl Integration

Querydsl cung cấp type-safe queries:

## 6.5 Querydsl Integration

Querydsl là một framework mạnh mẽ cho phép bạn xây dựng các truy vấn type-safe (an toàn kiểu) trong Java. Thay vì viết các chuỗi truy vấn JPQL hoặc Native SQL dễ bị lỗi cú pháp và khó kiểm tra tại thời điểm biên dịch, Querydsl cho phép bạn xây dựng các truy vấn bằng cách sử dụng các đối tượng Java được tạo tự động (Q-classes) đại diện cho các Entity và thuộc tính của chúng. Điều này giúp phát hiện lỗi sớm hơn và cải thiện khả năng đọc code.

### 6.5.1 Dependencies

Để sử dụng Querydsl với Spring Data JPA, bạn cần thêm các dependency sau vào file `pom.xml` của dự án:

XML

```
<dependency>
  <groupId>com.querydsl</groupId>
  <artifactId>querydsl-jpa</artifactId>
  <version>5.0.0</version> <!-- Sử dụng phiên bản phù hợp với dự án của bạn -->
</dependency>
<dependency>
  <groupId>com.querydsl</groupId>
  <artifactId>querydsl-apt</artifactId>
  <version>5.0.0</version> <!-- Phải khớp với phiên bản querydsl-jpa -->
  <scope>provided</scope> <!-- Scope provided vì nó chỉ cần thiết trong quá trình biên dịch để tạo Q-classes -->
</dependency>

<!-- Cấu hình plugin để tự động tạo Q-classes -->
<build>
  <plugins>
    <plugin>
      <groupId>com.mysema.maven</groupId>
      <artifactId>apt-maven-plugin</artifactId>
      <version>1.1.3</version> <!-- Phiên bản của plugin -->
      <executions>
        <execution>
          <goals>
            <goal>process</goal>
          </goals>
          <configuration>
            <outputDirectory>target/generated-
```

```

sources/java</outputDirectory> <!-- Thư mục chứa Q-classes được tạo -->

<processor>com.querydsl.apt.jpa.JPAAnnotationProcessor</processor> <!--
Processor cho JPA -->
    </configuration>
    </execution>
</executions>
<dependencies>
    <dependency>
        <groupId>com.querydsl</groupId>
        <artifactId>querydsl-apt</artifactId>
        <version>5.0.0</version>
    </dependency>
</dependencies>
</plugin>
</plugins>
</build>

```

### Giải thích:

- **querydsl-jpa** : Dependency chính cho việc tích hợp Querydsl với JPA. Nó cung cấp các lớp và tiện ích cần thiết để xây dựng truy vấn.
- **querydsl-apt** : Đây là một annotation processor. Nó được sử dụng trong quá trình biên dịch để tự động tạo ra các lớp **Q-classes** (ví dụ: **QUser** , **QProduct** ) từ các Entity JPA của bạn. Các **Q-classes** này là các đối tượng type-safe đại diện cho các Entity và thuộc tính của chúng, cho phép bạn xây dựng truy vấn một cách an toàn.
- **<scope>provided</scope>** : Chỉ ra rằng dependency này chỉ cần thiết trong quá trình biên dịch và sẽ không được đóng gói vào file JAR cuối cùng của ứng dụng.
- **apt-maven-plugin** : Plugin Maven này được cấu hình để chạy **querydsl-apt** annotation processor. Nó đảm bảo rằng các **Q-classes** được tạo ra tự động mỗi khi bạn biên dịch dự án. Thư mục **target/generated-sources/java** là nơi các **Q-classes** sẽ được tạo ra.

### 6.5.2 Repository với Querydsl

Để cho phép Repository của bạn sử dụng Querydsl, interface repository cần kế thừa **QuerydslPredicateExecutor<T>** . Interface này cung cấp các phương thức để thực thi các truy vấn dựa trên **Predicate** của Querydsl.

```

public interface UserRepository extends JpaRepository<User, Long>,
QuerydslPredicateExecutor<User> {
    // Inherited methods:
    // Optional<T> findOne(Predicate predicate); // Tìm một Entity dựa trên
    Predicate.
    // Iterable<T> findAll(Predicate predicate); // Tìm tất cả các Entity dựa
    trên Predicate.
    // Iterable<T> findAll(Predicate predicate, Sort sort); // Tìm và sắp
    xếp.
    // Page<T> findAll(Predicate predicate, Pageable pageable); // Tìm và
    phân trang.
    // long count(Predicate predicate); // Đếm số lượng Entity khớp với
    Predicate.
    // boolean exists(Predicate predicate); // Kiểm tra sự tồn tại của Entity
    khớp với Predicate.
}

```

### Giải thích:

- **QuerydslPredicateExecutor<T>** : Interface này cung cấp các phương thức để thực thi các truy vấn Querydsl. **T** là kiểu Entity mà Repository này quản lý.
- **Predicate** : Là một interface trong Querydsl đại diện cho một điều kiện truy vấn (tương tự như **Specification** trong JPA Criteria API). Bạn sẽ xây dựng các **Predicate** bằng cách sử dụng các **Q-classes** .
- **Kế thừa**: Bằng cách kế thừa **QuerydslPredicateExecutor** , **UserRepository** của bạn sẽ có sẵn các phương thức **findAll** , **findOne** , **count** , **exists** nhận vào một **Predicate** làm tham số, cho phép bạn thực hiện các truy vấn Querydsl.

### 6.5.3 Sử dụng Querydsl

Sau khi đã cấu hình Querydsl và Repository của bạn kế thừa **QuerydslPredicateExecutor** , bạn có thể bắt đầu xây dựng và thực thi các truy vấn Querydsl trong các lớp Service hoặc Business Logic của mình.

Java

```

@Service
public class UserService {

```

```

@Autowired
private UserRepository userRepository;

public List<User> findUsers(String name, Integer minAge, String status) {
    QUser user = QUser.user; // Lấy instance của Q-class cho Entity User.

    BooleanBuilder builder = new BooleanBuilder(); // Tạo một
    BooleanBuilder để xây dựng các điều kiện truy vấn.

    if (name != null) {
        builder.and(user.name.containsIgnoreCase(name)); // Thêm điều
        kiện tìm kiếm theo tên (không phân biệt hoa thường).
    }

    if (minAge != null) {
        builder.and(user.age.goe(minAge)); // Thêm điều kiện tìm kiếm
        theo tuổi (lớn hơn hoặc bằng).
    }

    if (status != null) {
        builder.and(user.status.eq(status)); // Thêm điều kiện tìm kiếm
        theo trạng thái (khớp chính xác).
    }

    return (List<User>) userRepository.findAll(builder); // Thực thi truy
    vấn với Predicate đã xây dựng.
}

public Page<User> findUsersWithRoles(String roleName, Pageable pageable)
{
    QUser user = QUser.user;
    QRole role = QRole.role;

    // Xây dựng Predicate để tìm User có Role cụ thể.
    BooleanExpression predicate = user.roles.any().name.eq(roleName); //
    `any()` cho phép truy vấn trên các phần tử của một collection.

    return userRepository.findAll(predicate, pageable); // Thực thi truy
    vấn với phân trang.
}
}

```

## Giải thích:

- QUser user = QUser.user;** : QUser là lớp Querydsl được tạo tự động cho Entity User .  
 QUser.user là một instance tĩnh của lớp này, cho phép bạn truy cập các thuộc tính của

Entity một cách type-safe (ví dụ: `user.name` , `user.age` ).

- **BooleanBuilder** : Một lớp tiện ích của Querydsl giúp bạn xây dựng các `Predicate` phức tạp bằng cách kết hợp các điều kiện với `and()` và `or()` . Nó rất hữu ích khi bạn có nhiều điều kiện tùy chọn.
- **`user.name.containsIgnoreCase(name)`** : Ví dụ về cách xây dựng điều kiện truy vấn type-safe. `containsIgnoreCase` tương đương với `LIKE %value%` và bỏ qua chữ hoa/thường.
- **`user.age.goe(minAge)`** : `goe` là viết tắt của "greater than or equal" (lớn hơn hoặc bằng).
- **`user.status.eq(status)`** : `eq` là viết tắt của "equals" (bằng).
- **`user.roles.any().name.eq(roleName)`** : Ví dụ về truy vấn trên các mối quan hệ. `any()` được sử dụng khi bạn muốn kiểm tra một điều kiện trên bất kỳ phần tử nào trong một collection (ví dụ: một User có nhiều Roles, bạn muốn tìm User có bất kỳ Role nào đó).
- **`userRepository.findAll(builder)`** : Các phương thức `findAll` trong `QuerydslPredicateExecutor` nhận một `Predicate` (hoặc `BooleanBuilder` vì nó implement `Predicate` ) làm tham số để thực thi truy vấn.
- **Lợi ích:** Querydsl mang lại sự an toàn kiểu cho các truy vấn, giúp phát hiện lỗi cú pháp ngay tại thời điểm biên dịch thay vì runtime. Nó cũng cải thiện khả năng đọc và bảo trì code, đặc biệt cho các truy vấn phức tạp.

## 6.6 Performance Tuning

Performance Tuning (tối ưu hóa hiệu suất) là một khía cạnh quan trọng khi làm việc với JPA và Spring Data JPA. Việc truy vấn dữ liệu không hiệu quả có thể dẫn đến các vấn đề về hiệu suất nghiêm trọng, đặc biệt là khi làm việc với lượng lớn dữ liệu hoặc các mối quan hệ phức tạp.

### 6.6.1 N+1 Query Problem

N+1 Query Problem là một vấn đề hiệu suất phổ biến xảy ra khi bạn tải một danh sách các Entity cha, và sau đó cho mỗi Entity cha, bạn lại thực hiện một truy vấn riêng biệt để tải các Entity con liên quan (thường là các mối quan hệ `OneToMany` hoặc `ManyToMany` với

`FetchType.LAZY` ). Điều này dẫn đến N+1 truy vấn (1 truy vấn để lấy các Entity cha và N truy vấn để lấy các Entity con), gây lãng phí tài nguyên và làm chậm ứng dụng.

Java

```
// Problem: N+1 queries
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "department", fetch = FetchType.LAZY) // Mỗi quan
    hệ LAZY loading.
    private List<Employee> employees = new ArrayList<>();

    // constructors, getters, setters
}

@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;

    // constructors, getters, setters
}

// Ví dụ gây ra N+1:
// Khi bạn tải tất cả các Department, sau đó lặp qua từng Department để truy
cập danh sách Employee.
// 1 truy vấn để lấy tất cả Department.
// N truy vấn (với N là số Department) để lấy Employee cho từng Department.
// List<Department> departments = departmentRepository.findAll();
// for (Department dept : departments) {
//     dept.getEmployees().size(); // Truy cập employees sẽ kích hoạt truy
vấn riêng.
// }

// Solution 1: JOIN FETCH
// Sử dụng JOIN FETCH trong JPQL để tải các Entity liên quan trong cùng một
```

truy vấn.

```
@Query("SELECT d FROM Department d JOIN FETCH d.employees")  
List<Department> findAllWithEmployees();
```

*// Solution 2: @EntityGraph*

*// Sử dụng @EntityGraph để chỉ định các mối quan hệ cần được tải cùng với Entity chính.*

```
@EntityGraph(attributePaths = {"employees"}) // Chỉ định thuộc tính  
'employees' cần được tải ngay lập tức.  
List<Department> findAll();
```

*// Solution 3: Custom @EntityGraph (sử dụng NamedEntityGraph)*

*// Định nghĩa NamedEntityGraph trong Entity để tái sử dụng cấu hình EntityGraph.*

```
@Entity
```

```
@NamedEntityGraph(  
    name = "Department.employees", // Tên của EntityGraph.  
    attributeNodes = @NamedAttributeNode("employees") // Chỉ định thuộc tính  
    cần tải.  
)
```

```
public class Department {  
    // ... (như trên)  
}
```

```
public class Department {  
    // ... (như trên)  
}
```

```
}
```

```
public class Department {  
    // ... (như trên)  
}
```

```
public class Department {  
    // ... (như trên)  
}
```

```
}
```

```
@EntityGraph("Department.employees") // Sử dụng NamedEntityGraph đã định  
nghĩa.  
List<Department> findByStatus(String status);
```

```
List<Department> findByStatus(String status);
```

## Giải thích:

- **N+1 Query Problem:** Xảy ra khi bạn tải một tập hợp các Entity (N), và sau đó cho mỗi Entity đó, bạn lại thực hiện một truy vấn riêng biệt để tải các mối quan hệ **LAZY** của nó. Ví dụ, nếu bạn có 100 **Department** và mỗi **Department** có một danh sách **Employee** được tải **LAZY**, việc truy cập `getEmployees()` cho từng **Department** sẽ dẫn đến 101 truy vấn (1 cho **Department** và 100 cho **Employee**).
- **JOIN FETCH**: Đây là cách hiệu quả nhất để giải quyết vấn đề N+1. Bằng cách sử dụng **JOIN FETCH** trong truy vấn JPQL, bạn yêu cầu JPA tải các Entity liên quan ( **employees** ) cùng với Entity chính ( **Department** ) trong một truy vấn duy nhất. Điều này giúp giảm đáng kể số lượng truy vấn đến cơ sở dữ liệu.
- **@EntityGraph**: Một annotation của JPA cho phép bạn định nghĩa một biểu đồ các thuộc tính (mối quan hệ) của Entity cần được tải cùng với Entity chính. Khi bạn sử dụng



`@EntityGraph` trên một phương thức repository, Spring Data JPA sẽ tự động tạo truy vấn `JOIN FETCH` (hoặc tương đương) để tải các thuộc tính được chỉ định. Nó cung cấp một cách khai báo để giải quyết N+1 mà không cần viết JPQL thủ công.

- **`attributePaths`** : Chỉ định tên của các thuộc tính (mối quan hệ) cần được tải ngay lập tức.
- **`@NamedEntityGraph`** : Cho phép bạn định nghĩa một `@EntityGraph` trong lớp Entity và đặt tên cho nó. Điều này giúp tái sử dụng cùng một cấu hình EntityGraph trên nhiều phương thức repository khác nhau, tránh trùng lặp code. Bạn tham chiếu đến nó bằng tên trong `@EntityGraph` trên phương thức repository.

## 6.6.2 Batch Processing

Batch Processing (xử lý theo lô) là kỹ thuật thực hiện nhiều thao tác INSERT, UPDATE, hoặc DELETE trong một lần gửi đến cơ sở dữ liệu thay vì gửi từng thao tác một. Điều này giúp giảm đáng kể số lượng round-trip giữa ứng dụng và cơ sở dữ liệu, từ đó cải thiện hiệu suất cho các tác vụ xử lý dữ liệu lớn.

Java

```
@Repository // Đánh dấu là một Spring Repository.
public class UserBatchRepository {

    @PersistenceContext // Inject EntityManager để tương tác trực tiếp với
    JPA.
    private EntityManager entityManager;

    @Transactional // Đảm bảo toàn bộ thao tác batch chạy trong một
    transaction.
    public void batchInsertUsers(List<User> users) {
        int batchSize = 20; // Kích thước lô: số lượng Entity sẽ được xử lý
        trong mỗi batch.

        for (int i = 0; i < users.size(); i++) {
            entityManager.persist(users.get(i)); // Đưa Entity vào
            Persistence Context.

            // Nếu đã đạt đến batchSize hoặc là Entity cuối cùng, thực hiện
            flush và clear.
            if (i > 0 && i % batchSize == 0) {
                entityManager.flush(); // Đồng bộ hóa Persistence Context với
                cơ sở dữ liệu (thực hiện các INSERT).
```

```

        entityManager.clear(); // Xóa tất cả các Entity khỏi
        Persistence Context để giải phóng bộ nhớ.
    }
}

entityManager.flush(); // Đảm bảo các Entity còn lại trong batch cuối
cùng cũng được flush.
entityManager.clear(); // Giải phóng bộ nhớ sau khi hoàn tất.
}

@Transactional
public void batchUpdateUsers(List<User> users) {
    int batchSize = 20;

    for (int i = 0; i < users.size(); i++) {
        entityManager.merge(users.get(i)); // Hợp nhất Entity vào
        Persistence Context (nếu nó là detached).

        if (i > 0 && i % batchSize == 0) {
            entityManager.flush();
            entityManager.clear();
        }
    }

    entityManager.flush();
    entityManager.clear();
}
}

```

## Giải thích:

- **EntityManager.persist(entity)** : Đưa một Entity mới vào Persistence Context. Entity này sẽ được INSERT vào cơ sở dữ liệu khi `flush()` được gọi.
- **EntityManager.merge(entity)** : Hợp nhất một Entity (có thể là detached) vào Persistence Context. Nếu Entity đã tồn tại trong DB, nó sẽ được UPDATE khi `flush()` được gọi.
- **EntityManager.flush()** : Buộc Persistence Context đồng bộ hóa với cơ sở dữ liệu. Điều này có nghĩa là tất cả các thao tác INSERT, UPDATE, DELETE đang chờ xử lý trong Persistence Context sẽ được gửi đến cơ sở dữ liệu. Việc gọi `flush()` định kỳ giúp giảm lượng bộ nhớ mà Persistence Context sử dụng và đảm bảo các thay đổi được ghi vào DB.
- **EntityManager.clear()** : Xóa tất cả các Entity khỏi Persistence Context. Điều này rất quan trọng trong Batch Processing để giải phóng bộ nhớ và ngăn chặn Persistence Context trở

nên quá lớn, gây ra vấn đề về hiệu suất. Sau khi `clear()`, các Entity đã được quản lý trước đó sẽ trở thành detached.

- **batchSize** : Kích thước của mỗi lô. Việc chọn `batchSize` phù hợp là rất quan trọng; quá nhỏ sẽ vẫn gây ra nhiều round-trip, quá lớn có thể gây ra vấn đề về bộ nhớ hoặc timeout.
- **@Transactional** : Toàn bộ thao tác batch nên được bọc trong một transaction để đảm bảo tính nguyên tử (atomic) của các thao tác. Nếu có lỗi xảy ra giữa chừng, toàn bộ batch có thể được rollback.
- **Cấu hình JDBC Batching**: Để Batch Processing hoạt động hiệu quả, bạn cũng cần cấu hình JDBC driver để hỗ trợ batching. Trong Spring Boot, bạn có thể thêm các thuộc tính sau vào `application.properties` :
  - `hibernate.jdbc.batch_size` : Số lượng câu lệnh SQL được nhóm lại trong một batch.
  - `hibernate.order_inserts`, `hibernate.order_updates` : Đảm bảo Hibernate sắp xếp các câu lệnh INSERT/UPDATE để tối ưu hóa batching.
  - `hibernate.jdbc.batch_versioned_data` : Cho phép batching cho các Entity có `@Version`.

### 6.6.3 Second Level Cache

Second Level Cache (L2 Cache) là một lớp cache nằm giữa ứng dụng và cơ sở dữ liệu. Mục tiêu của nó là giảm số lượng truy vấn đến cơ sở dữ liệu bằng cách lưu trữ các Entity đã được tải trong bộ nhớ dùng chung (shared memory) của ứng dụng. Khi một Entity được yêu cầu, JPA/Hibernate sẽ kiểm tra L2 Cache trước khi truy vấn cơ sở dữ liệu. Nếu Entity có trong cache, nó sẽ được trả về ngay lập tức, giúp cải thiện đáng kể hiệu suất.

#### Plain Text

```
# Enable second level cache: Kích hoạt Second Level Cache.
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
# Cấu hình nhà cung cấp cache (ví dụ: EhCache, Redis, Caffeine).
spring.jpa.properties.hibernate.cache.region.factory_class=org.hibernate.cache.ehcache.EhCacheRegionFactory
# Kích hoạt Query Cache: Lưu trữ kết quả của các truy vấn (không phải Entity).
spring.jpa.properties.hibernate.cache.use_query_cache=true
```

Java

```
@Entity
@Cacheable // Đánh dấu Entity này có thể được cache ở Second Level Cache.
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE) // Chỉ định chiến lược
cache (ví dụ: READ_WRITE).
public class User {
    // Entity definition
}

// Repository với query cache: Sử dụng @QueryHints để kích hoạt query cache
cho một truy vấn cụ thể.
public interface UserRepository extends JpaRepository<User, Long> {
    @QueryHints(@QueryHint(name = "org.hibernate.cacheable", value = "true"))
    List<User> findByStatus(String status);
}
```

## Giải thích:

- **hibernate.cache.use\_second\_level\_cache=true** : Thuộc tính này kích hoạt Second Level Cache trong Hibernate.
- **hibernate.cache.region.factory\_class** : Chỉ định implementation của cache provider mà bạn muốn sử dụng (ví dụ: **EhCacheRegionFactory** cho EhCache, **RedisCacheRegionFactory** cho Redis). Bạn cần thêm dependency của cache provider tương ứng vào **pom.xml** .
- **hibernate.cache.use\_query\_cache=true** : Kích hoạt Query Cache. Query Cache lưu trữ ID của các Entity được trả về bởi một truy vấn cụ thể, không phải chính các Entity đó. Khi truy vấn được thực thi lại, Hibernate sẽ kiểm tra Query Cache. Nếu kết quả có trong cache, nó sẽ lấy các ID Entity từ cache và sau đó tải các Entity đó từ Second Level Cache (nếu có) hoặc từ cơ sở dữ liệu.
- **@Cacheable** : Annotation này được đặt trên lớp Entity để chỉ định rằng các instance của Entity này có thể được lưu trữ trong Second Level Cache.
- **@Cache(usage = CacheConcurrencyStrategy.READ\_WRITE)** : Chỉ định chiến lược đồng bộ hóa cache. Các chiến lược phổ biến:
  - **READ\_ONLY** : Dành cho dữ liệu không thay đổi. Hiệu suất cao nhất.

- `READ_WRITE` : Dành cho dữ liệu thường xuyên được đọc và có thể thay đổi. Đảm bảo tính nhất quán.
- `NONSTRICT_READ_WRITE` : Tương tự `READ_WRITE` nhưng không đảm bảo tính nhất quán mạnh mẽ. Có thể chấp nhận dữ liệu cũ trong một khoảng thời gian ngắn.
- `TRANSACTIONAL` : Đảm bảo tính nhất quán cao nhất trong môi trường transaction.
- `@QueryHint(name = "org.hibernate.cacheable", value = "true")` : Annotation này được sử dụng trên các phương thức repository để kích hoạt Query Cache cho một truy vấn cụ thể. Điều này chỉ có tác dụng nếu `hibernate.cache.use_query_cache` đã được bật.
- **Lợi ích:** Giảm tải cho cơ sở dữ liệu, cải thiện đáng kể thời gian phản hồi của ứng dụng, đặc biệt cho các truy vấn thường xuyên được thực hiện trên dữ liệu ít thay đổi.